

Generative AI Usage and Disclosure: EECS 581 Assignment 1

General Disclosure:

ChatGPT's GPT6-Astra model was used on 09/26/26 to write the C program for this assignment.

Code attribution and prompt:

The entire 185-line program was generated by the GPT6-Astra model. No modifications were made because the program passed all 27 student-written test cases. The following prompt was used to generate the program:

"Create a C program that reads a line of text and extracts a single valid IPv4 address (optionally followed by a port number) embedded anywhere within the line of text. Only digits, periods, and colons can be part of a valid token (all other characters are skipped). A candidate token must match the address grammar (specified below) in full. No partial matches or truncations to make a valid address are allowed.

Address Format:

An address is 4 octets separated by periods. Each octet is 1 to 3 digits long, and can have a value between 0 and 255 (both inclusive). No leading zero is allowed unless the value is exactly 0. An optional :port may follow the 4th octet. This port number is 1 to 5 digits long, has a value between 0 and 65535 (both inclusive), and cannot have a leading zero unless the value is exactly 0. If a colon is present, the port number must be fully valid or the entire match (including address) is rejected.

Function prototype:

The function will return whether a valid address was found, and deliver the address and port through 2 additional parameters. Here is the prototype:

```
// Returns 1 if a valid address was found, 0 otherwise.
```

```
// On success: *outAddress holds the 32-bit value, and *outPort holds the port number or -1 if no port was present.
```

```
// On failure: *outAddress is set to 0 and *outPort is set to -1.
```

```
int extractIPv4(const char* str, unsigned long* outAddress, int* outPort);
```

Built-in functions and libraries that are NOT ALLOWED:

Any string-to-number conversion function: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions. Any address-parsing library function: inet_aton, inet_pton, inet_addr, or equivalents. Any regular-expression facility

(std::regex, POSIX regex.h, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.

Note: Standard character-classification functions (isdigit, etc.) are fine to use.

Acceptance Requirements:

Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation. Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.

Output and Program Requirements:

On success, print: "Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)" where N is the 32-bit decimal value and P is the port number or the literal text none. On failure, print: "Invalid input: no valid IPv4 address found." Input loop: main continuously prompts the user for input until the user enters END (case-sensitive), then prints "Program terminated." and exits. Extraction function: implement extractIPv4 exactly as prototyped above. All digit accumulation must be done by hand. Display: main receives the result from extractIPv4 and formats the output exactly as specified above."

Verification Statement:

I, Peter Barybin, confirm that I analyzed the model's output and understand every line of the submitted code. The program has been tested with 27 test cases and works fully as intended. There are no known bugs or unexpected behaviors. The test cases were crafted thoroughly to account for all possible edge cases.